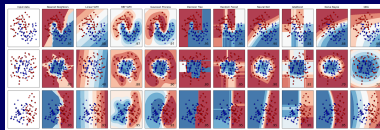


Árboles, ensembles, boosting moderno y optimización bayesiana

Aprendizaje Automático · Parte 2



Andrés Vásquez Restrepo

avasquezr3@eafit.edu.co

SI7009 - 1 (5553)

Universidad EAFIT

Contenido

1. CART: impurezas, splits y pruning

Pruning

2. Bias-variance

3. Ensembles como remedios arquitectónicos

Voting Classifiers

4. Gradient boosting moderno

5. Balanceo: nivel de datos y de modelo

6. Validación cruzada para HPO

7. Optimización bayesiana

8. Cierre

Dónde quedamos

Parte 1

- métricas, costos y threshold;
- workflow, EDA y baseline serio;
- validación cruzada estratificada, agrupada y temporal;
- ROC, PR y calibración;
- búsqueda de hiperparámetros: grid, random, halving y nested CV.

Idea clave: **Pregunta puente**

Ya sabemos evaluar y buscar configuraciones. ¿Qué modelos vale la pena optimizar, y cómo buscar mejor cuando cada entrenamiento es caro?

El mapa de la sesión

1. Árboles de decisión (CART), pruning y bias-variance.
2. Ensembles: voting, bagging y boosting.
3. Boosting moderno: XGBoost, LightGBM y CatBoost.
4. Balanceo a nivel de datos y de modelo.
5. CV para HPO y optimización bayesiana con Optuna.
6. Hands-on y defensa técnica del primer experimento.

La historia completa es una cadena: problema → datos → CV → métrica → threshold → decisión.

CART: impurezas, splits y pruning

Por qué hablar de árboles en esta sesión

El árbol es el objeto más simple para entender particiones, complejidad, sobreajuste, estabilidad y ensembles.

- No buscamos hacer benchmarking de algoritmos.
- Buscamos entender por qué un modelo flexible puede ser frágil.
- El árbol es el *learner* base de bagging y boosting.

¿Qué es un Decision Tree?

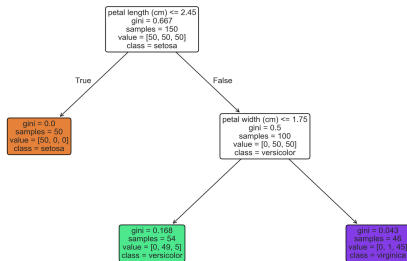
Un árbol de decisión es un modelo de **aprendizaje supervisado** que realiza predicciones mediante una *estructura jerárquica de reglas de decisión*.

Componentes principales:

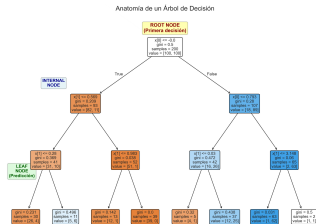
- **Nodo raíz:** primera decisión
- **Nodos internos:** reglas de división
- **Nodos hoja:** predicciones finales
- **Ramas:** caminos de decisión

Ventajas: interpretable, no requiere normalización, maneja features categóricas y numéricas.

Decision Tree on Iris Dataset (max_depth=2)



Anatomía de un árbol de decisión



Proceso de predicción:

1. Comenzar en el nodo raíz
2. Evaluar condición del nodo actual
3. Seguir rama correspondiente (True/False)
4. Repetir hasta alcanzar un nodo hoja
5. Retornar predicción del nodo hoja

CART: Algoritmo fundamental

CART (*Classification and Regression Trees*) construye árboles binarios mediante particiones recursivas que minimizan una función de costo.

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

- k : feature seleccionado para el split; t_k : umbral para dividir;
- G : medida de impureza (Gini o Entropy);
- m : número total de muestras; m_{left} , m_{right} : muestras en cada hijo.

CART: Algoritmo paso a paso

Procedimiento recursivo:

1. **Inicio:** Conjunto completo de datos en raíz
2. **Para cada nodo:**
 - Evaluar todos los features
 - Para cada feature, probar todos los *thresholds* posibles
 - Seleccionar (k, t_k) que minimiza $J(k, t_k)$
 - Crear dos nodos hijos con las particiones
3. **Recursión:** Repetir para cada hijo
4. **Criterio de parada:**
 - Profundidad máxima alcanzada
 - Número mínimo de muestras no cumplido
 - No hay reducción significativa de impureza
 - Nodo puro (impureza = 0)

Propiedades de CART

Ventajas:

- Árboles binarios (fácil implementación)
- Maneja *features* numéricos y categóricos
- Solución determinística
- No requiere normalización
- Robusto a *outliers*

Desventajas:

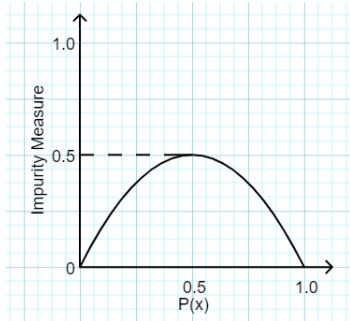
- *Greedy algorithm* (no garantiza óptimo global)
- Sensible a rotaciones de datos
- Propenso a *overfitting*
- Inestable (pequeños cambios → árbol diferente)
- Probabilidades mal calibradas (hojas puras → 0 o 1)

Complejidad computacional

Entrenamiento: $O(n \cdot m \log_2(m))$ donde n = features, m = muestras

Gini Impurity: Definición y derivación

La impureza de Gini mide la **probabilidad** de clasificar incorrectamente una muestra seleccionada aleatoriamente.



Máximo binario: Gini 0,5, entropía 1,0. Ambas miden qué tan mezclado está un nodo; los árboles buscan grupos más homogéneos.

Gini Impurity: Definición y derivación

Fórmula:

$$G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

Donde:

- $p_{i,k}$: proporción de muestras de clase k en el nodo i
- n : número total de clases
- Rango: $[0, 1 - \frac{1}{n}]$
- $G_i = 0$ indica pureza perfecta (todas las muestras de una clase)

Gini Impurity: Interpretación probabilística

Derivación intuitiva:

Si seleccionamos dos muestras al azar del nodo i :

- Probabilidad de que la primera sea clase k : $p_{i,k}$
- Probabilidad de que la segunda NO sea clase k : $(1 - p_{i,k})$
- Probabilidad de clasificación incorrecta para clase k : $p_{i,k}(1 - p_{i,k})$

Sumando sobre todas las clases:

$$G_i = \sum_{k=1}^n p_{i,k}(1 - p_{i,k}) = \sum_{k=1}^n p_{i,k} - \sum_{k=1}^n p_{i,k}^2 = 1 - \sum_{k=1}^n p_{i,k}^2$$

Casos extremos

Máxima impureza (distribución uniforme): $G = 1 - \frac{1}{n}$

Mínima impureza (nodo puro): $G = 0$

Gini Impurity: Ejemplo numérico

Escenario: Nodo con 54 muestras

- Clase 0 (setosa): 0 muestras $\rightarrow p_{i,0} = 0/54 = 0$
- Clase 1 (versicolor): 49 muestras $\rightarrow p_{i,1} = 49/54 \approx 0,907$
- Clase 2 (virginica): 5 muestras $\rightarrow p_{i,2} = 5/54 \approx 0,093$

Cálculo de Gini:

$$G_i = 1 - (0^2 + 0,907^2 + 0,093^2)$$

$$G_i = 1 - (0 + 0,823 + 0,009) = 1 - 0,832 = 0,168$$

Interpretación

$G = 0,168$ indica impureza moderada. El nodo está dominado por versicolor (90.7 %) pero tiene algunas virginicas (9.3 %).

Entropy: Definición desde la teoría de la información

Entropía: mide el grado de *incertidumbre* o *desorden* en un conjunto de datos.

- Concepto introducido en 1948 por *Claude Shannon* en su artículo *A Mathematical Theory of Communication*.
- La entropía cuantifica la cantidad *promedio de información necesaria* para describir una variable aleatoria.
- Cuanto mayor es la entropía, mayor es la incertidumbre; una buena división reduce esa entropía, creando nodos más puros.

Entropy: Definición desde teoría de información

Fórmula:

$$H_i = - \sum_{k=1}^n p_{i,k} \log_2(p_{i,k})$$

Donde:

- $p_{i,k}$: proporción de clase k en nodo i
- $\log_2$: logaritmo base 2 (medida en bits)
- Convención: $0 \log_2(0) = 0$
- Rango: $[0, \log_2(n)]$
- $H_i = 0$ cuando hay una sola clase

Interpretación de Shannon

Número promedio de bits necesarios para codificar la información.

Entropy: Ejemplo numérico y comparación

Mismo escenario anterior: 54 muestras (0 setosa, 49 versicolor, 5 virginica)

$$H_i = -(0 \cdot \log_2(0) + 0,907 \cdot \log_2(0,907) + 0,093 \cdot \log_2(0,093))$$

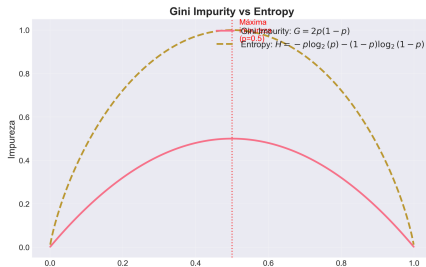
$$H_i = -(0 + 0,907 \cdot (-0,141) + 0,093 \cdot (-3,426))$$

$$H_i = -(-0,128 - 0,319) = 0,447 \text{ bits}$$

Comparación Gini vs Entropy:

■ Gini: $G = 0,168$

■ Entropy: $H = 0,447$



Information Gain: Reducción de entropía

El **Information Gain** mide cuánta incertidumbre se reduce al hacer un split.

$$IG = H_{\text{parent}} - \sum_{j \in \{\text{left}, \text{right}\}} \frac{n_j}{n} H_j$$

- H_{parent} : entropía del nodo padre; H_j : entropía del hijo j ;
- n_j : muestras en el hijo j ; n : muestras totales.

Objetivo

Maximizar IG equivale a minimizar la entropía ponderada de los hijos.

Gini vs Entropy: ¿Cuál usar?

Gini Impurity:

- Más rápido computacionalmente
- No requiere logaritmos
- Tiende a aislar clase más frecuente
- Default en *scikit-learn*
- Rango: $[0, 0,5]$ (binario)

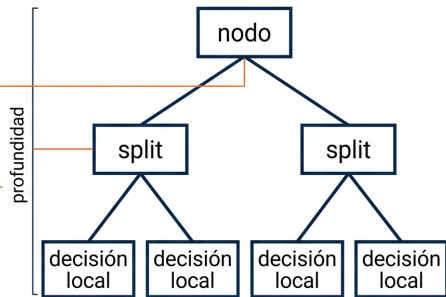
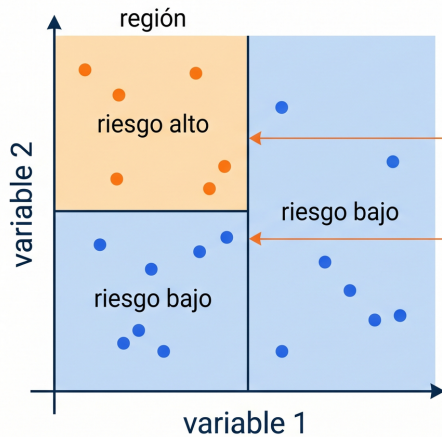
Entropy:

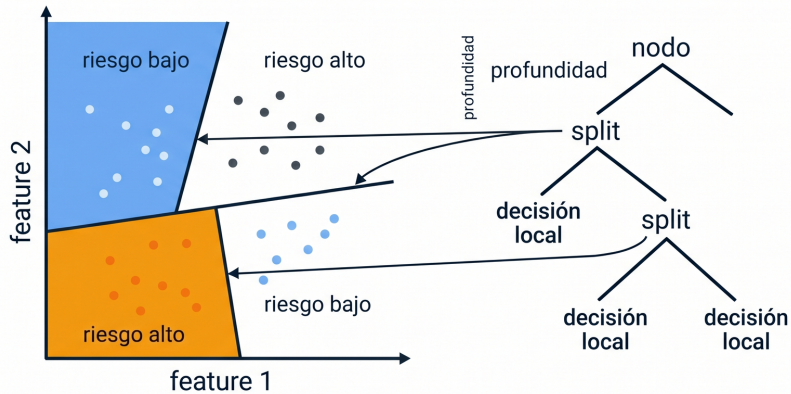
- Computacionalmente más costoso
- Base teórica más sólida
- Tiende a árboles más balanceados
- Penaliza más fuertemente impureza
- Rango: $[0, 1]$ (binario)

Recomendación práctica

En la mayoría de casos producen árboles similares. Usar Gini por defecto (velocidad). Probar Entropy si se buscan árboles más balanceados. Con desbalance, ambas quedan dominadas por la clase mayoritaria: `class_weight` modifica las proporciones p_k .

Pruning: controlar la complejidad del árbol



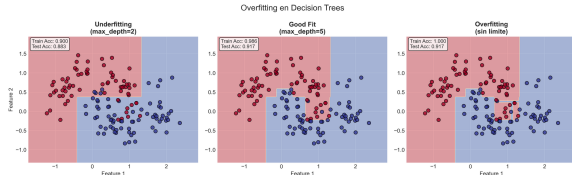


El problema del overfitting

Sin restricciones, un árbol de decisión puede crecer hasta memorizar perfectamente el conjunto de entrenamiento.

Síntomas de overfitting:

- Árbol muy profundo
- Hojas con pocas muestras
- Error training ≈ 0
- Error test $\gg$ Error training
- Decisiones *boundary* muy irregulares



Modelos no paramétricos

Decision trees son modelos no paramétricos: el número de parámetros crece con los datos

Pre-pruning: hiperparámetros de regularización

Pre-pruning: Detener crecimiento durante construcción

- `max_depth`: Profundidad máxima del árbol
 - Típico: 3–10 para evitar overfitting
 - ∞ permite memorización completa
- `min_samples_split`: Mínimo de muestras para dividir nodo
 - Típico: 2–20
 - Valores altos $\rightarrow$ árboles más simples
- `min_samples_leaf`: Mínimo de muestras en hoja
 - Asegura generalización
 - Evita hojas con muestras individuales
- `max_leaf_nodes`: Número máximo de hojas
 - Control directo de complejidad
 - Alternativa a `max_depth`

Post-pruning por costo-complejidad

Se deja crecer el árbol y luego se podan las ramas que no justifican su complejidad.

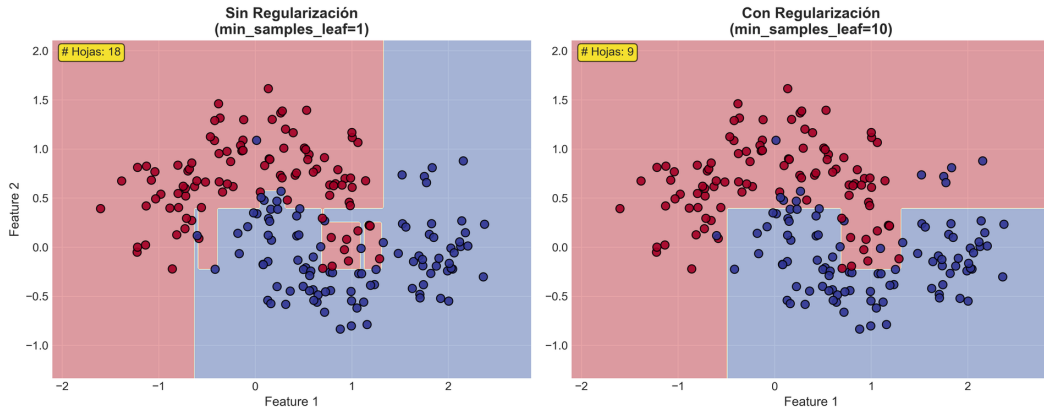
$$R_{\alpha}(T) = R(T) + \alpha |T|$$

- $R(T)$: error (impureza total) del árbol; $|T|$: número de hojas;
- α grande $\rightarrow$ árbol más pequeño;
- `ccp_alpha` se elige **con CV**, como cualquier hiperparámetro.

```
1 path = DecisionTreeClassifier().cost_complexity_pruning_path(X_train, y_train)
2 grid = GridSearchCV(DecisionTreeClassifier(), {"ccp_alpha": path.ccp_alphas},
3               cv=skf, scoring="average_precision")
```

Efecto de la regularización

Efecto de Regularización (min_samples_leaf)



Izquierda: Sin regularización (*overfitting*)

Derecha: Con $\text{min_samples_leaf}=4$ (mejor generalización)

Bias–variance: complejidad y error

Profundidad: potencia y fragilidad

Árbol poco profundo

- reglas más simples;
- mayor estabilidad;
- posible underfitting;
- lectura más fácil.

Árbol muy profundo

- reglas muy específicas;
- alta sensibilidad al dato;
- posible overfitting;
- peor generalización.

La profundidad aumenta capacidad, pero también puede aumentar fragilidad.

Bias-variance: dos formas de equivocarse

Concepto	Qué significa	Síntoma típico
Alto bias	el modelo es demasiado rígido	falla incluso en train
Alta variance	el modelo es demasiado sensible al dataset observado	funciona en train y falla fuera de muestra

- Bias-variance no es un adorno teórico: es lenguaje para pensar complejidad, estabilidad y error.

Descomposición bias–variance

Con $y = f(x) + \varepsilon$ y esperanza sobre datasets de entrenamiento:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{ruido}}$$

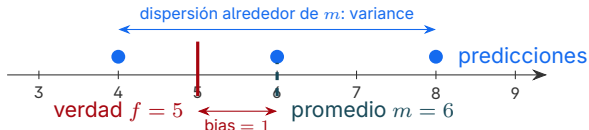
Modelo	Bias	Variance
Árbol profundo	bajo	alta
<i>Stump</i> (profundidad 1)	alto	baja
Árbol podado (<code>ccp_alpha</code>)	compromiso	compromiso

Idea clave: **Conexión con la CV**

La variance se observa en la dispersión del score entre folds.

Descomposición bias-variance: ejemplo numérico

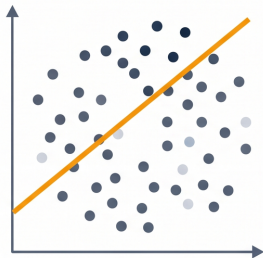
Sin ruido ($\sigma^2 = 0$). Verdad en x : $f(x) = 5$. Se entrena 3 veces con muestras distintas y los modelos predicen 4, 6 y 8.



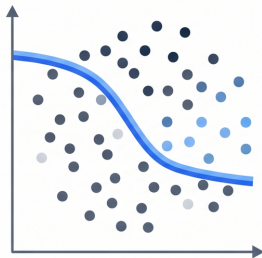
Término	Cálculo	Valor
Error total	$[(5 - 4)^2 + (5 - 6)^2 + (5 - 8)^2] / 3 = (1 + 1 + 9) / 3$	3,67
Bias²	$(m - f)^2 = (6 - 5)^2$	1
Variance	$[(4 - 6)^2 + (6 - 6)^2 + (8 - 6)^2] / 3 = (4 + 0 + 4) / 3$	2,67
Bias² + Variance	$1 + 2,67$	3,67

El error se parte exacto: lejanía del promedio a la verdad (bias²) + dispersión de los modelos (variance).

alto bias

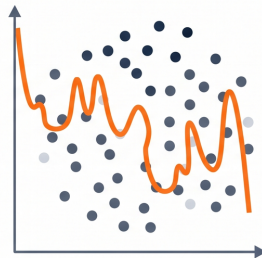


modelo rígido
underfitting



modelo balanceado
generalización

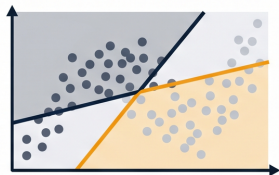
alta variance



modelo inestable
overfitting

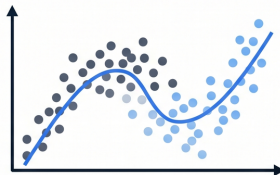
complejidad

árbol poco profundo



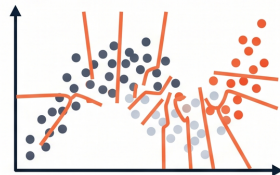
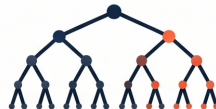
alto bias
modelo rígido
underfitting

profundidad moderada



modelo balanceado
generalización

árbol muy profundo



alta variance
modelo inestable
overfitting

← aumenta la profundidad →
más flexibilidad, más riesgo de inestabilidad

Ensembles: remedios arquitectónicos para bias y variance

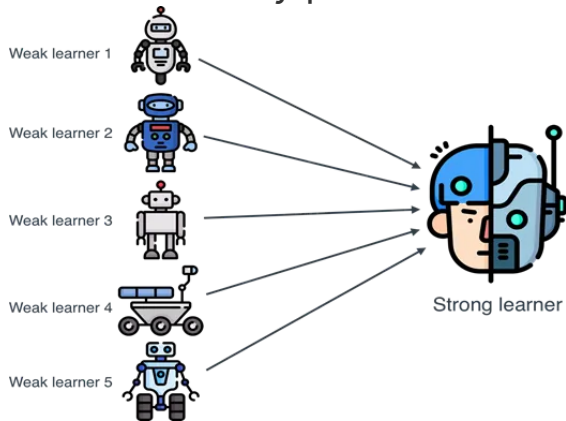
¿Qué es Ensemble Learning?

Principio fundamental: Combinar múltiples modelos (*weak learners*) para crear un predictor más robusto y preciso.

Analogía:

- Pregunta compleja a 1000 personas
- Promedia sus respuestas
- Resultado mejor que respuesta de experto

Wisdom of the Crowd



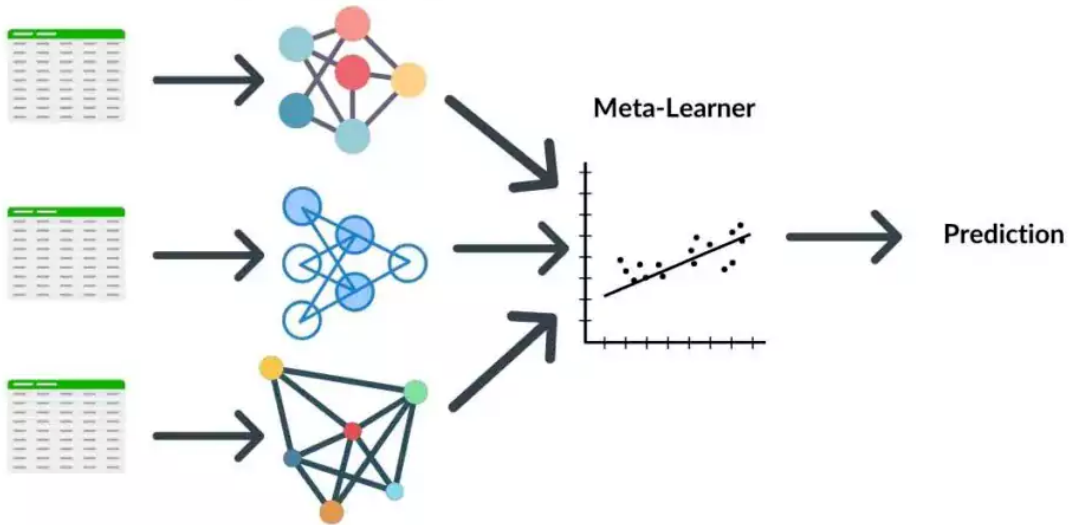
¿Qué es Ensemble Learning?

Fórmula general:

$$\hat{y}_{\text{ensemble}} = f(\hat{y}_1, \hat{y}_2, \dots, \hat{y}_M)$$

Donde f es agregación (promedio, votación, ponderado)

Different Models



¿Por qué funciona Ensemble Learning?

Requisito clave: Los modelos deben ser DIVERSOS (errores no correlacionados)

Matemática básica:

Para M modelos independientes con error ϵ cada uno:

$$\text{Error}_{\text{ensemble}} \approx \frac{\epsilon}{\sqrt{M}} \quad (\text{idealización})$$

Con correlación ρ entre modelos: $\text{Var} = \rho \sigma^2 + \frac{1-\rho}{M} \sigma^2$: la diversidad importa.

Ley de los grandes números

Con suficientes *weak learners* independientes, el *ensemble* converge a la predicción correcta

Hey robots!,
is this a cat?



Great, it seems that
an ensemble of robots
works better!

Yes!



No!



Yes!



Yes!

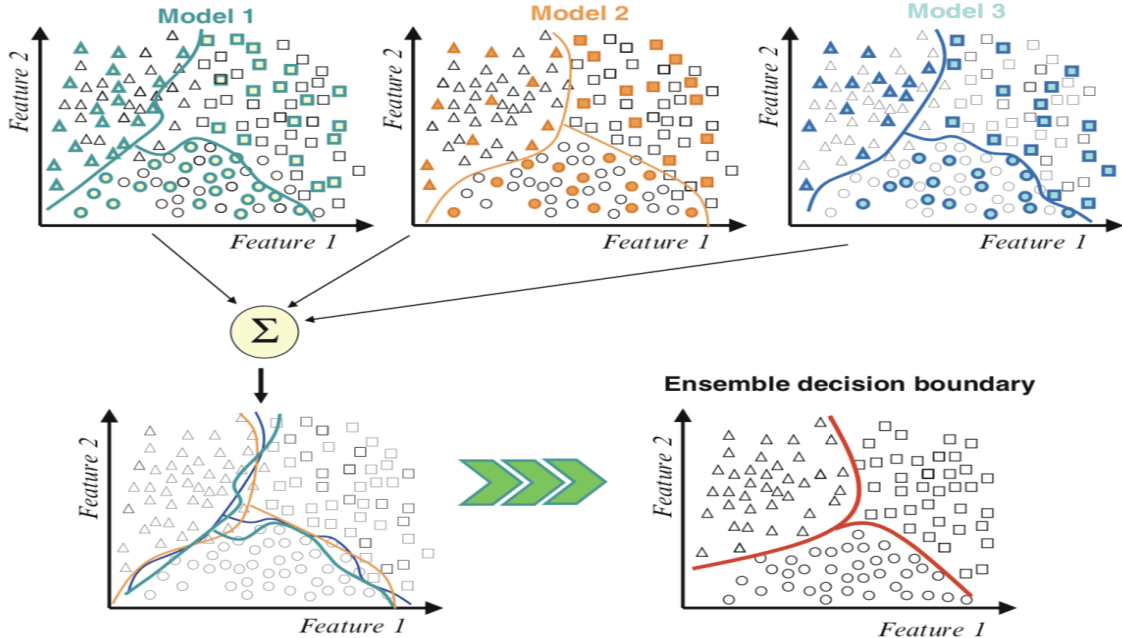


Yes!



No!





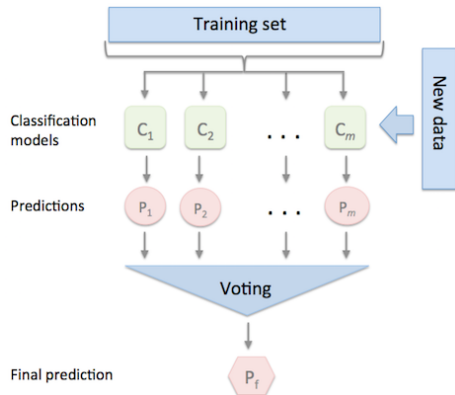
Votación: El Ensemble Más Simple

Voting Classifiers: Concepto

Combina predicciones de múltiples modelos diferentes mediante votación.

Idea central

Entrenar varios clasificadores distintos (SVM, Logistic Regression, Decision Tree, etc.) y agregar sus predicciones.



Voting Classifiers: Concepto

Hard Voting (clasificación):

$$\hat{y} = \text{mode}(\hat{y}_1, \hat{y}_2, \dots, \hat{y}_M)$$

Soft Voting (probabilidades):

$$\hat{y} = \arg \max_k \sum_{m=1}^M p_m(y = k|x)$$

Regresión (promedio):

$$\hat{y} = \frac{1}{M} \sum_{m=1}^M \hat{y}_m$$

Hard vs Soft Voting

Hard Voting:

- Cada modelo vota por una clase
- Gana la clase con más votos
- Simple pero puede perder información

Ejemplo:

- SVM: clase 1
- LR: clase 1
- DT: clase 0
- RF: clase 1

Resultado: clase 1 (3 votos vs 1)

Soft Voting:

- Promedia probabilidades
- Gana clase con mayor probabilidad promedio
- Usa confianza de cada modelo

Ejemplo:

- SVM: $P(1)=0.6$
- LR: $P(1)=0.55$
- DT: $P(1)=0.45$
- RF: $P(1)=0.7$

Resultado: $P(1)=0.575 \rightarrow$ clase 1

Requisito para Soft Voting

Todos los clasificadores deben tener método `predict_proba()`

Requisito crítico: Diversidad

Problema: Si los clasificadores cometen los mismos errores, voting no ayuda.

Estrategias para diversidad:

- Usar algoritmos muy diferentes (SVM, trees, logistic regression, etc.)
- Entrenar en diferentes subconjuntos de features
- Usar ...
- Entre

Voting Classifier: Diversidad de Modelos



Bagging: estabilizar modelos inestables

Si muchos árboles son sensibles al dato, agregarlos reduce la inestabilidad.

- Se entrenan varios modelos sobre muestras bootstrap del dataset y se promedian.
- **Bagging ataca la variance:** promedia árboles profundos de bajo bias.
- **Random Forest** = bagging + submuestreo aleatorio de features en cada split.
- `BalancedRandomForestClassifier` (imblearn) combina bagging y balanceo.

Boosting: corregir errores de forma secuencial

Boosting construye una secuencia donde cada etapa intenta corregir los errores anteriores.

	Bagging	Boosting
Learner base	profundo, bajo bias	débil, alto bias
Entrenamiento	en paralelo	secuencial
Reduce	variance	bias
Riesgo	poco beneficio si los árboles están correlacionados	sobreajuste con demasiadas etapas

Frontera

XGBoost, LightGBM y CatBoost se desarrollan en la sección siguiente; su búsqueda de hiperparámetros con Optuna, al final de hoy.

(Opcional) Multiclase sin deep learning

Modelos multiclase nativos

Árboles, Random Forest y gradient boosting predicen K clases directamente.

Reducción a problemas binarios

- **One-vs-Rest:** K clasificadores "clase k vs. resto"; gana el de mayor score.
- **One-vs-One:** $K(K-1)/2$ clasificadores por pares; gana la clase más votada.

```
1 from sklearn.multiclass import (  
2     OneVsRestClassifier, OneVsOneClassifier)  
3 ovr = OneVsRestClassifier(  
4     DecisionTreeClassifier(max_depth=4))
```

Cuidado

Codificar clases como números y usar un regresor impone un orden que no existe. Evaluar con macro-F1.

Gradient boosting moderno: XGBoost, LightGBM y CatBoost

Gradient boosting: sumar árboles que corrigen errores

Cada árbol nuevo h_m se ajusta a lo que el modelo actual todavía hace mal:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

- h_m aprende el **gradiente negativo** de la pérdida: en regresión, el residuo $y - F_{m-1}(x)$; en log-loss, $y - \hat{p}_i$
- cada árbol es pequeño (alto bias): boosting **reduce bias** etapa a etapa;
- η (*learning rate*) encoge cada paso.

Idea clave: Regla práctica

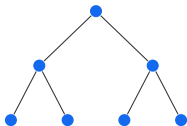
η pequeño + más árboles generaliza mejor, pero cuesta más. El número de árboles se elige con **early stop-ping**, no a mano.

XGBoost, LightGBM y CatBoost

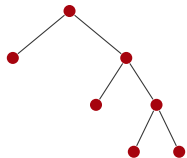
	XGBoost	LightGBM	CatBoost
Idea clave	objetivo regularizado (λ, γ) y aproximación de segundo orden	histogramas + crecimiento por hojas; muestreo GOSS	<i>ordered boosting</i> contra el sesgo del target
Crecimiento	por niveles (<code>max_depth</code>)	por hojas (<code>num_leaves</code>)	árboles simétricos (<code>depth</code>)
Catóricas	requiere encoding (soporte nativo reciente)	nativas por partición óptima	nativas con <i>ordered target statistics</i>
Fortaleza	robusto, muy usado, buen manejo de faltantes	muy rápido en datos grandes	buenos defaults; muchas catóricas
Cuidado	muchos hiperparámetros	sobreajusta con pocos datos	entrenamiento más lento en CPU

Los tres implementan la misma idea; cambian cómo construyen cada árbol y cómo tratan catóricas. La teoría de cada uno se desarrolla en la sesión 2.

Crecimiento por niveles frente a por hojas



Por niveles (XGBoost): divide todas las hojas de un nivel antes de bajar.



Por hojas (LightGBM): divide la hoja que más reduce la pérdida.

- por niveles: árboles balanceados; se controla con `max_depth`;
- por hojas: con el mismo número de hojas suele reducir más la pérdida, pero crea ramas profundas que sobreajustan con pocos datos;
- se controla con `num_leaves` y `min_child_samples`;
- CatBoost: árboles simétricos, mismo split en todo un nivel.

Regla

$\text{num_leaves} < 2^{\text{max_depth}}$ para no anular el límite de profundidad.

Regularización y submuestreo

Palanca	XGBoost	LightGBM	CatBoost	Efecto
Tasa de aprendizaje	learning_rate	learning_rate	learning_rate	pasos más pequeños, más árboles
Número de árboles	n_estimators	n_estimators	iterations	fijar alto y cortar con early stopping
Complejidad del árbol	max_depth	num_leaves	depth	menos capacidad, menos variance
Mínimo por hoja	min_child_weight	min_child_samples	min_data_in_leaf	evita hojas con muy pocos casos
Penalización L1 / L2	reg_alpha, reg_lambda	reg_alpha, reg_lambda	l2_leaf_reg	encoge los valores de las hojas
Submuestreo de filas	subsample	subsample*	subsample	cada árbol ve otra muestra
Submuestreo de columnas	colsample_bytree	colsample_bytree	rsm	árboles menos correlacionados
Desbalance	scale_pos_weight	scale_pos_weight	auto_class_weights	pondera la clase rara

El submuestreo trae a boosting una idea de bagging: introduce diversidad y reduce variance. Nombres según la API de scikit-learn de cada librería. *En LightGBM requiere `subsample_freq > 0`.

Balanceo: nivel de datos y de modelo

El problema del desbalance de clases

¿Cuándo ocurre?

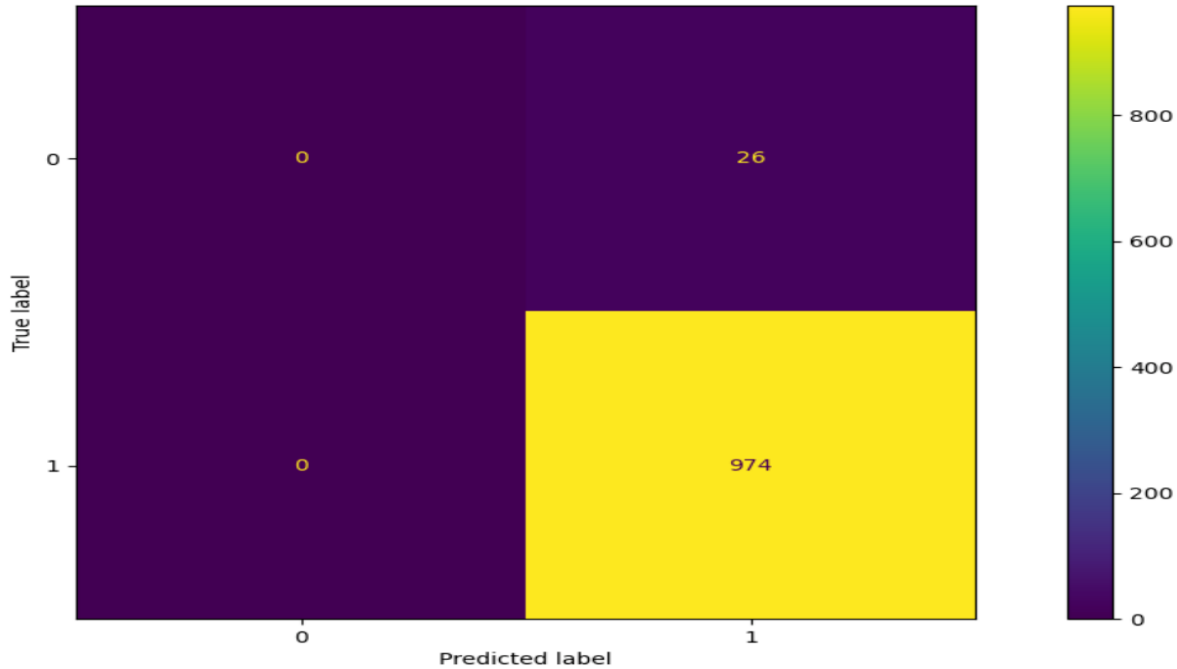
- fraude: $\approx 0,1-0,2$ % positivos;
- diagnóstico de enfermedades raras;
- churn: ≈ 5 % abandona;
- detección de anomalías;
- multiclase: en *Amazon reviews* domina 5 estrellas.

¿Por qué es problemático?

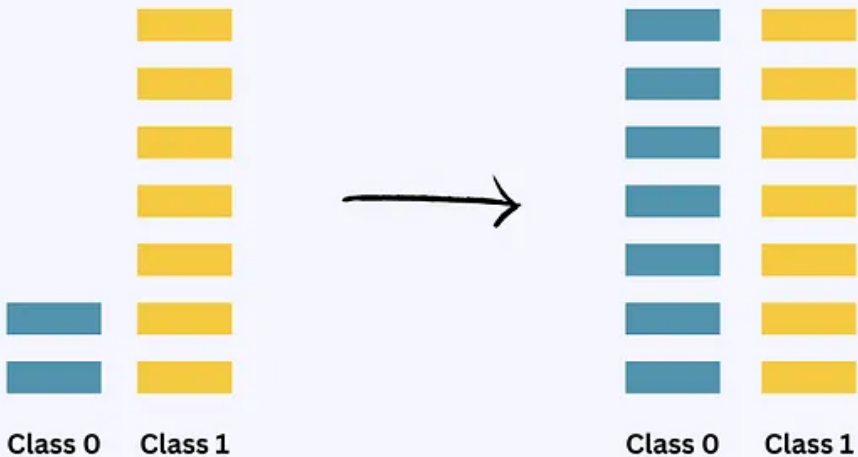
- el modelo trivial obtiene alta accuracy;
- los algoritmos se sesgan hacia la mayoría;
- la clase rara queda ignorada.

Idea clave: Recordatorio

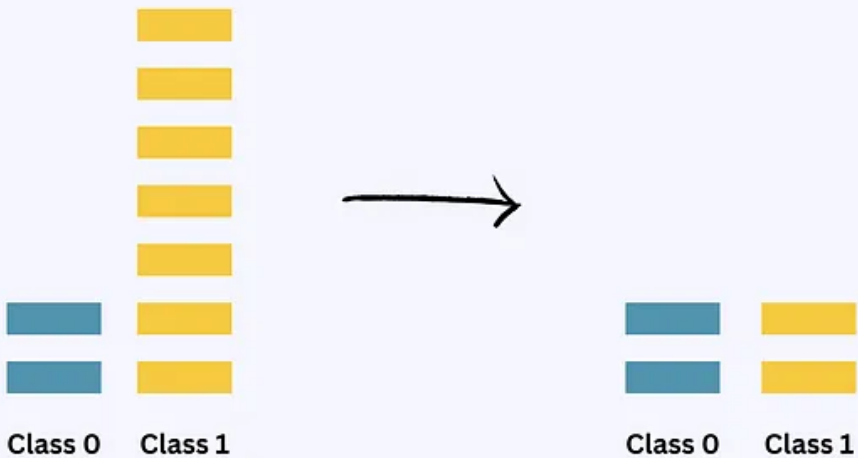
99,8 % de aciertos y recall de fraude igual a cero: ya sabemos medirlo. Ahora, ¿cómo lo mejoramos?



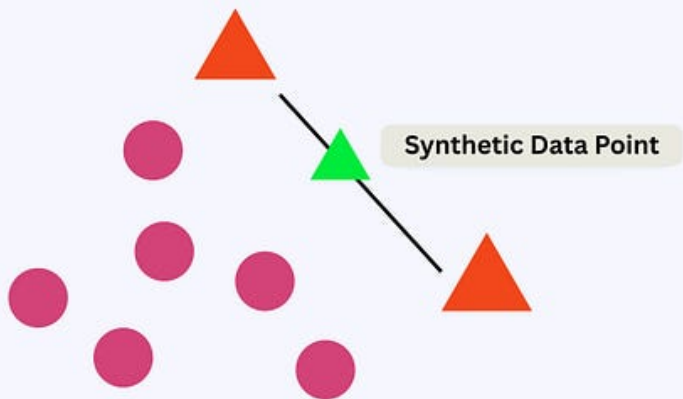
OVERSAMPLING



UNDERSAMPLING



SMOTE

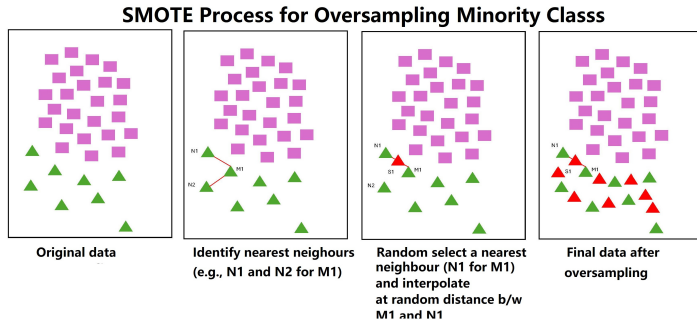


Estrategias para datos desbalanceados

Nivel de datos:

- Ajustes **sólo** en *train*; dev/test conservan distribución.
- **Undersampling**: Submuestreo (reducción) de la clase mayoritaria.
- **Oversampling**: Remuestreo (aumento) de clases minoritarias.
- Generación sintética
 - **SMOTE**: Synthetic Minority Oversampling
 - **Borderline-SMOTE**: sintéticos solo cerca de la frontera de decisión
 - **ADASYN**: Adaptive Synthetic Sampling

Estrategias para datos desbalanceados



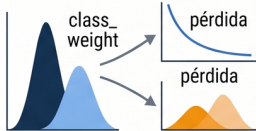
Nivel de algoritmo:

- **Class weights:** penalizar más los errores en la minoría
- **Balanced loss:** pérdida ponderada por clase
- **Focal loss** (conceptual): concentrar el aprendizaje en los casos difíciles

ESTRATEGIAS PARA DESBALANCE DE CLASES: INTERVENCIONES METODOLÓGICAS

class_weight

Ajuste de pesos en la pérdida para priorizar clase minoritaria



Ajuste de pesos en la pérdida para priorizar clase minoritaria

beneficio:

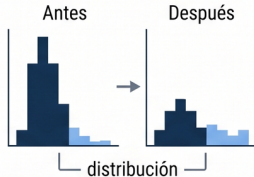
Simple,
eficiente

riesgo:

Sobreajuste a
patrones raros

undersampling

Reducción de muestras de la clase mayoritaria



beneficio:

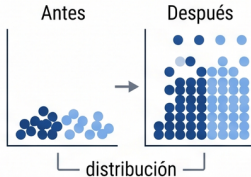
Reduce sesgo
mayoritario,
acelera
entrenamiento

riesgo:

Pérdida de
información

oversampling

Réplica de muestras de la clase minoritaria existentes



beneficio:

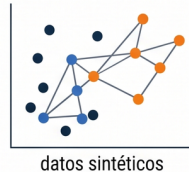
Retiene información
mayoritaria,
aumenta visibilidad
minoritaria

riesgo:

Alto riesgo de
sobreajuste,
réplicas exactas

SMOTE

Generación de nuevos ejemplos
sintéticos entre vecinos
minoritarios cercanos



beneficio:

Crea diversidad
de ejemplos,
reduce réplicas
exactas

riesgo:

Generación
ruidosa

Fair comparson la etrategion (e.g., cross-validation,
or unified testing on an independiente hold-out set)

validación
no es receta

Entramier
al modelo



Cálculo de
métricos



Análisis de
resultados

class_weight: balanced loss

En lugar de cambiar los datos, se cambia cuánto pesa equivocarse en cada clase.

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n w_{y_i} \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right] \quad w_k = \frac{n}{K n_k}$$

- n : observaciones; K : clases; n_k : observaciones de la clase k .
- `class_weight="balanced"` implementa exactamente estos pesos.
- En árboles, los pesos modifican las proporciones p_k usadas en Gini y Entropy.

class_weight: qué gana y qué arriesga

Puede ayudar a

- aumentar atención sobre la clase rara;
- reducir falsos negativos;
- mantener todos los datos originales;
- crear una primera intervención simple.

Cuidado

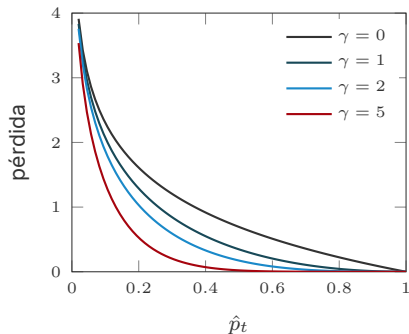
No garantiza mejor decisión. Puede mejorar recall y empeorar precision, por eso debe evaluarse con threshold y métrica correcta. Además distorsiona $\hat{p}$: recalibrar antes de usar probabilidades.

Focal loss (conceptual)

$$\mathcal{L}_{FL} = -\alpha_t (1 - \hat{p}_t)^\gamma \log \hat{p}_t$$

$\hat{p}_t = \hat{p}$ si $y = 1$; $\hat{p}_t = 1 - \hat{p}$ si $y = 0$.

- $(1 - \hat{p}_t)^\gamma$ reduce el peso de los ejemplos fáciles (la mayoría de negativos obvios);
- $\gamma = 0$ recupera la cross-entropy ponderada;
- α_t balancea las clases;
- en boosting se usa como pérdida personalizada.



Lin et al. (2017), *Focal Loss for Dense Object Detection*.

Under/oversampling: cambiar lo que el modelo ve

Undersampling

- reduce casos de la clase mayoritaria;
- acelera entrenamiento;
- puede perder información útil;
- depende mucho de cómo se selecciona.

Oversampling

- aumenta casos de clase minoritaria;
- puede mejorar señal positiva;
- puede inducir overfitting;
- exige validación cuidadosa.

Cambiar la distribución de entrenamiento exige proteger la validación.

SMOTE no es magia

SMOTE crea ejemplos sintéticos de la clase minoritaria interpolando entre observaciones cercanas.

Variantes: **Borderline-SMOTE** (solo cerca de la frontera) y **ADASYN** (más sintéticos donde la clase es más difícil).

Puede ser útil cuando

- la clase positiva tiene pocos ejemplos;
- hay señal local aprovechable;
- se usa dentro del pipeline correcto;
- se valida sin contaminar el test.

Riesgo metodológico

Si se aplica antes del split o fuera de la validación cruzada, puede introducir leakage y producir una comparación demasiado optimista.

Flujo incorrecto: rebalancing antes de dividir

Datos completos $\rightarrow$ SMOTE / oversampling $\rightarrow$ split $\rightarrow$ score optimista

Por qué falla

La validación deja de representar el mundo real y puede recibir información derivada de observaciones que debían quedar fuera del entrenamiento.

Flujo defendible: rebalancing dentro de validación

Split / CV → rebalancing solo en train → validación en distribución natural

```
1 from imblearn.pipeline import Pipeline
2 from imblearn.over_sampling import SMOTE
3
4 pipe = Pipeline([("smote", SMOTE(random_state=42)),
5                  ("tree", DecisionTreeClassifier(max_depth=4))])
6 cross_val_score(pipe, X, y, cv=StratifiedKFold(5, shuffle=True, random_state=42),
7                  scoring="average_precision")
```

Importante

Rebalancear solo en train, nunca en validación ni en test. La intervención es parte del pipeline evaluado.

Comparar estrategias de balanceo

Estrategia	Nivel	Qué cambia	Efecto típico	Riesgo
Random undersampling	datos	elimina mayoritarios	↑ recall, ↓ precision	pierde información
Random oversampling	datos	duplica minoritarios	↑ recall	overfitting a duplicados
SMOTE	datos	interpola minoritarios	↑ recall	ruido; leakage fuera del fold
Borderline-SMOTE	datos	interpola cerca de la frontera	↑ recall en zona difícil	amplifica ruido de frontera
ADASYN	datos	más sintéticos donde es difícil	↑ recall	sensible a outliers
Class weights / balanced loss	modelo	pondera la pérdida	↑ recall	puede ↓ precision
Focal loss	modelo	enfoca en casos difíciles	↑ en difíciles	hiperparámetro γ extra

Todas alteran $\hat{p}$: recalibrar o elegir τ por validación.

Discusión: ¿qué probaría primero?

Taller / tarea: **Caso: primera intervención contra desbalance**

Su baseline probabilístico detecta pocos fraudes. Debe proponer una primera estrategia para mejorar la lectura de la clase positiva sin destruir la validez del experimento.

Elija y justifique

- `class_weight`;
- `undersampling`;
- `oversampling`;
- SMOTE;
- focal loss (conceptual).

Debe declarar

- métrica principal;
- esquema de CV;
- threshold a revisar;
- riesgo metodológico.

Validación cruzada para optimización de hiperparámetros

La CV es la función objetivo de cada trial

Un *trial* de Optuna propone hiperparámetros; la CV devuelve el número que el optimizador intenta maximizar.

```
1 import optuna
2 from lightgbm import LGBMClassifier
3 skf = StratifiedKFold(5, shuffle=True, random_state=42)    # mismos folds en todos los trials
4
5 def objective(trial):
6     params = {"learning_rate": trial.suggest_float("learning_rate", 1e-3, 0.3, log=True),
7              "num_leaves": trial.suggest_int("num_leaves", 15, 255, log=True),
8              "min_child_samples": trial.suggest_int("min_child_samples", 10, 200)}
9     model = LGBMClassifier(n_estimators=500, **params)
10    return cross_val_score(model, X_train, y_train, cv=skf,
11                           scoring="average_precision").mean()
12
13 study = optuna.create_study(direction="maximize")
14 study.optimize(objective, n_trials=50)
```

- solo X_train: el test no participa en ningún trial;
- el objeto skf fijo hace comparables todos los trials.

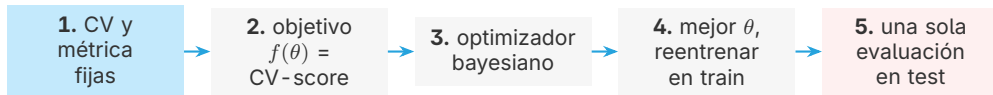
Elegir el tipo de fold según los datos

Situación	Fold	Por qué
clases desbalanceadas	StratifiedKFold	cada fold conserva la prevalencia de positivos
varias filas por cliente o tarjeta	StratifiedGroupKFold	un mismo grupo no aparece en train y validación
datos con cronología	TimeSeriesSplit (con gap)	se entrena con el pasado y se valida con el futuro
datos muy grandes o trials caros	holdout estratificado o 3 folds	menos costo por trial

Idea clave: **Criterio**

El fold debe imitar cómo llegarán los datos nuevos en producción. Si se elige mal, Optuna optimiza con precisión un score que no se va a repetir.

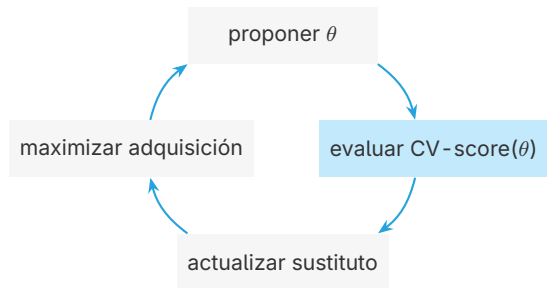
Por qué la CV debe definirse antes de la búsqueda bayesiana



- el optimizador solo ve el número que devuelve la CV: **si la CV tiene leakage, lo optimiza con mucha eficiencia;**
- cambiar folds o métrica a mitad del estudio vuelve incomparables los trials anteriores: el sustituto aprende de datos inconsistentes;
- una CV ruidosa (pocos positivos por fold) hace que el sustituto persiga ruido: estratificar o repetir la CV;
- muchos trials sobre la misma CV también sobreajustan a ella: por eso el test queda aparte.

Optimización bayesiana con Optuna

La idea: construir un modelo del score



- evaluar la CV es caro; evaluar el **sustituto** es casi gratis;
- el sustituto estima el score (y su incertidumbre) en configuraciones no probadas;
- la **función de adquisición** decide dónde vale la pena gastar la siguiente evaluación;
- cada trial mejora el sustituto: la búsqueda se concentra en zonas buenas.

Modelos sustitutos: procesos gaussianos y TPE

	Proceso gaussiano (GP)	TPE (Optuna por defecto)
Qué modela	$p(\text{score} \mid \theta)$: media $\mu(\theta)$ e incertidumbre $\sigma(\theta)$	$p(\theta \mid \text{bueno}) = \ell(\theta)$ y $p(\theta \mid \text{malo}) = g(\theta)$
Cómo elige	maximiza EI o UCB sobre μ y σ	separa los trials por un cuantil del score y elige θ que maximiza $\ell(\theta)/g(\theta)$
Espacios	continuos, pocas dimensiones	mixtos: enteros, categóricos, condicionales
Costo	crece como n^3 con los trials	crece suave; escala a muchos trials

Idea clave: Lectura de TPE

"Busca valores que aparecen seguido entre los mejores trials y poco entre los peores."

Funciones de adquisición: EI y UCB

Expected Improvement (EI)

$$\text{EI}(\theta) = \mathbb{E}[\max(f(\theta) - f^*, 0)]$$

¿Cuánto espero superar al mejor score actual f^* ? Premia μ alta y también σ alta.

Upper Confidence Bound (UCB)

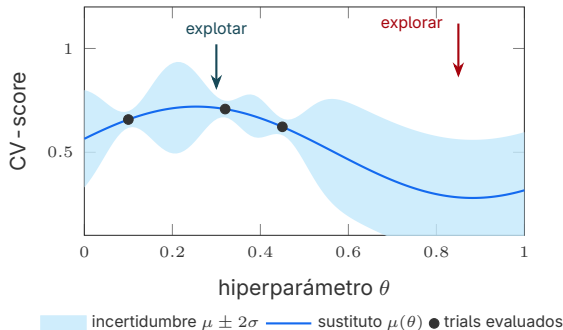
$$\text{UCB}(\theta) = \mu(\theta) + \kappa \sigma(\theta)$$

Optimista frente a la incertidumbre. κ grande explora más; κ pequeño explota más.

Ambas responden la misma pregunta

¿Dónde es más probable encontrar algo mejor que lo que ya tengo, teniendo en cuenta lo que no sé?

Exploración frente a explotación



- **explotar**: probar cerca del mejor trial, donde μ es alta y σ baja;
- **explorar**: probar donde no hay trials y σ es grande: ahí puede haber algo mejor;
- solo explotar se queda en un óptimo local; solo explorar es random search;
- Optuna empieza con trials aleatorios (`n_startup_trials`) antes de confiar en TPE.

Early stopping dentro de cada trial

```
1 import lightgbm as lgb
2 def objective(trial):
3     params = {"learning_rate": trial.suggest_float("learning_rate", 1e-3, 0.3, log=True),
4               "num_leaves": trial.suggest_int("num_leaves", 15, 255, log=True),
5               "subsample": trial.suggest_float("subsample", 0.5, 1.0), "subsample_freq": 1,
6               "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0),
7               "reg_lambda": trial.suggest_float("reg_lambda", 1e-3, 10, log=True)}
8     scores = []
9     for step, (tr, va) in enumerate(skf.split(X_train, y_train)):
10         model = LGBMClassifier(n_estimators=2000, **params) # alto: lo corta early stopping
11         model.fit(X_train.iloc[tr], y_train.iloc[tr],
12                 eval_set=[(X_train.iloc[va], y_train.iloc[va])],
13                 callbacks=[lgb.early_stopping(100, verbose=False)])
14         p = model.predict_proba(X_train.iloc[va])[:, 1]
15         scores.append(average_precision_score(y_train.iloc[va], p))
16     return np.mean(scores)
```

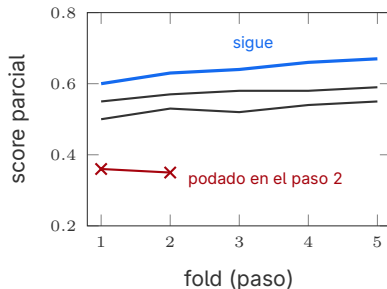
Early stopping para de agregar árboles cuando la pérdida de validación deja de mejorar: elige `n_estimators` dentro del trial. Usa el fold de validación, así que el score queda algo optimista: la evaluación final sigue siendo en test.

Pruning: cortar trials que no prometen

```
# dentro del bucle de folds:
trial.report(np.mean(scores), step)
if trial.should_prune():
    raise optuna.TrialPruned()

study = optuna.create_study(
    direction="maximize",
    sampler=optuna.samplers.TPESampler(seed=42),
    pruner=optuna.pruners.MedianPruner(
        n_startup_trials=5))
study.optimize(objective, n_trials=100)
```

MedianPruner: corta el trial si su score parcial queda bajo la mediana de los trials anteriores en el mismo paso.



Early stopping corta **árboles** dentro de un modelo; pruning corta **trials** completos.

Buenas prácticas con Optuna

Antes y durante

- fijar CV, métrica y semillas antes del primer trial;
- escala log para `learning_rate`, `num_leaves` y penalizaciones;
- presupuesto por número de trials o por tiempo (`timeout`);
- guardar el estudio (`storage`) para poder retomarlo.

Después

- revisar `plot_optimization_history` y `plot_param_importances`;
- si el mejor valor está en el borde del rango, ampliarlo;
- reentrenar con `study.best_params` en todo el train;
- elegir el `threshold` y calibrar en validación; evaluar una sola vez en test.

Idea clave: En una frase

La búsqueda bayesiana gasta mejor el presupuesto, pero solo es tan buena como la CV que optimiza.

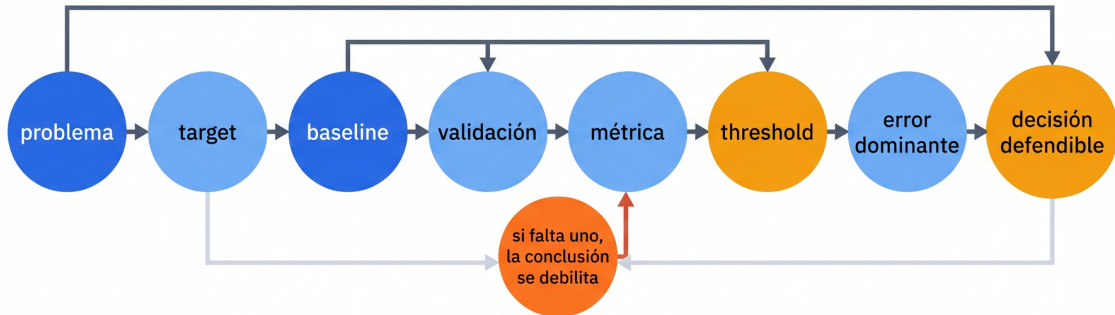
Hands-on de la parte 2

1. **SMOTE frente a `class_weight`** dentro de la CV: mismo pipeline, mismos folds; comparar PR-AUC y calibración.
2. **Optuna + LightGBM**: 50 trials con CV estratificada dentro del objetivo, early stopping y MedianPruner; comparar con el random search de la parte 1 al mismo presupuesto.

Herramienta: **Notebook**

[enlace al notebook de la sesión]

Cierre



Glosario mínimo: árboles, boosting y optimización bayesiana

- **CART** / `ccp_alpha`: árbol binario por impureza / poda por costo-complejidad.
- **Bias-variance**: rigidez frente a sensibilidad al dataset observado.
- **Gradient boosting**: suma de árboles que ajustan el gradiente de la pérdida.
- **Por niveles / por hojas**: `max_depth` (XGBoost) / `num_leaves` (LightGBM).
- **Early stopping**: dejar de agregar árboles cuando la validación no mejora.
- **Sustituto (GP, TPE)**: modelo barato del CV-score según los hiperparámetros.
- **El / UCB**: funciones de adquisición que equilibran explorar y explotar.
- **Pruning (Optuna)**: cortar trials cuyo score parcial queda bajo la mediana.

Glosario mínimo: ensembles y balanceo

- **Ensemble / voting:** combinar predicciones de varios modelos.
- **Bagging / Random Forest:** agregación sobre muestras bootstrap; reduce variance.
- **Boosting:** aprendizaje secuencial que corrige errores; reduce bias.
- **Under / oversampling:** reducir la mayoría / aumentar la minoría.
- **SMOTE, Borderline-SMOTE, ADASYN:** generación sintética de minoritarios.
- **Balanced loss:** pérdida ponderada por clase (`class_weight`).
- **Focal loss:** pérdida que reduce el peso de los ejemplos fáciles.

Preguntas que ya debe poder responder

1. ¿Por qué combinar modelos reduce el error y qué requisito exige?
2. ¿Bagging reduce bias o variance? ¿Y boosting?
3. ¿Qué riesgo aparece si aplica SMOTE antes del split?
4. ¿Qué diferencia hay entre SMOTE y Borderline-SMOTE?
5. ¿Qué hace `class_weight="balanced"` sobre la pérdida?
6. ¿Qué hace el factor $(1 - \hat{p}_t)^\gamma$ de la focal loss?
7. ¿Por qué hay que recalibrar después de rebalancear?

Preguntas que ya debe poder responder (II)

- 8. ¿Por qué un árbol profundo puede sobreajustar y cómo se poda?
- 9. ¿Qué diferencia hay entre crecimiento por niveles y por hojas?
- 10. ¿Qué palancas de regularización tiene LightGBM y qué efecto tiene cada una?
- 11. ¿Por qué la CV debe definirse antes de la búsqueda bayesiana?
- 12. ¿Qué hace la función de adquisición y qué significa explorar frente a explotar?
- 13. ¿En qué se diferencian early stopping y pruning de Optuna?

La frase técnica que debe sobrevivir

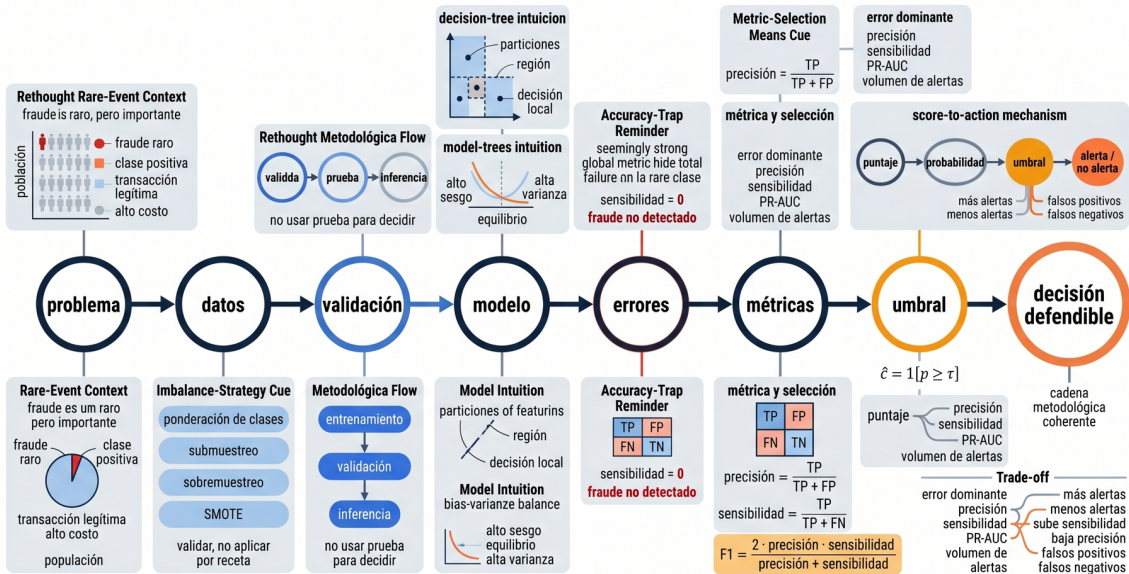
Un modelo no merece confianza porque su score se vea alto.

Merece confianza cuando el problema fue bien formulado, el baseline fue serio, la métrica fue correcta, la validación cruzada fue defendible y sin leakage, las probabilidades están calibradas y el threshold fue razonado.

En Aprendizaje Automático, no basta con entrenar modelos:
hay que defender decisiones de Machine Learning.

Idea clave: **Transición**

La próxima sesión profundiza en la teoría de XGBoost, LightGBM, CatBoost y otros modelos de boosting, sobre la misma CV y la misma búsqueda con Optuna.



si falta un eslabón, la conclusión se debilita

Gracias por su atención

¿Preguntas, comentarios o discusión?



Aprendizaje Automático
SI7009 · Universidad EAFIT

